

# Software Requirements Specification (SRS)

## TCS — Traffic Congestion System

|                   |                                 |
|-------------------|---------------------------------|
| Document Version  | 1.0                             |
| Date              | August 8, 2026                  |
| Prepared By       | Harsh Raj                       |
| Institution       | IITE, Indus University          |
| Project Title     | TCS — Traffic Congestion System |
| Document Standard | IEEE 830-1998 (SRS)             |

*Real-time AI-powered traffic monitoring using YOLOv8 and ByteTrack  
Python Backend | React + TypeScript Frontend | Flask REST API*

| Document Version  | 1.0                             |
|-------------------|---------------------------------|
| Date              | August 8, 2026                  |
| Prepared By       | Harsh Raj                       |
| Institution       | IITE, Indus University          |
| Project Title     | TCS — Traffic Congestion System |
| Document Standard | IEEE 830-1998 (SRS)             |

## Table of Contents

1. Introduction
  - 1.1 Purpose
  - 1.2 Scope
  - 1.3 Definitions, Acronyms, and Abbreviations
  - 1.4 References
  - 1.5 Overview
2. Overall Description
  - 2.1 Product Perspective
  - 2.2 Product Functions
  - 2.3 User Classes and Characteristics
  - 2.4 Operating Environment
  - 2.5 Design and Implementation Constraints
  - 2.6 Assumptions and Dependencies
3. System Architecture
  - 3.1 High-Level Architecture Diagram
  - 3.2 Module Decomposition
  - 3.3 Data Flow Diagram
4. Specific Requirements
  - 4.1 Functional Requirements
  - 4.2 Non-Functional Requirements
  - 4.3 External Interface Requirements
5. System Features (Detailed)
  - 5.1 Vehicle Detection & Classification
  - 5.2 Multi-Object Tracking
  - 5.3 Region of Interest (ROI) Filtering
  - 5.4 Congestion Detection Engine
  - 5.5 Email Alert System
  - 5.6 Desktop Notification System
  - 5.7 Geo-Location Service

- 5.8 Congestion Image Capture
  - 5.9 Analytics Graph Generation
  - 5.10 Real-Time Dashboard (Web Frontend)
  - 5.11 Live Video Streaming
  - 5.12 Camera Control API
6. Database / Data Storage Design
  7. User Interface Design
    - 7.1 Dashboard Layout
    - 7.2 UI Components
    - 7.3 Design System & Theming
  8. API Specification
  9. Technology Stack
  10. Module-Wise File Mapping
  11. Testing Strategy
  12. Deployment & Installation
  13. Risk Analysis
  14. Future Enhancements
  15. Appendices

## 1. Introduction

---

### 1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive and detailed description of the TCS — Traffic Congestion System. This document defines the functional and non-functional requirements, system architecture, interface specifications, design constraints, and operational characteristics of the system.

This SRS is intended for:

- Developers — to guide implementation and understand system architecture
- Academic Evaluators — to assess the project scope, complexity, and technical rigor
- Project Supervisors — to validate that the system meets its stated objectives
- Future Maintainers — to understand the codebase structure for extensions and bug fixes

### 1.2 Scope

The Traffic Congestion System (TCS) is a real-time, AI-powered traffic monitoring and alerting application. It performs the following high-level functions:

1. Real-time vehicle detection from live webcam or pre-recorded video feeds using the YOLOv8 deep learning model.

2. Multi-object tracking using the ByteTrack algorithm to assign persistent unique IDs to vehicles across consecutive video frames.
3. Congestion analysis based on configurable vehicle-count thresholds and sustained time durations.
4. Automated alerting via email notifications (with congestion evidence images), desktop notifications, and audible beep alerts.
5. Geo-location tagging of congestion events using IP-based location services, with Google Maps links embedded in alerts.
6. Analytics generation including time-series vehicle count graphs saved as PNG files.
7. Web-based real-time dashboard built with React + TypeScript (Vite), displaying live video feeds streamed from a Flask backend, with live metrics, status indicators, and camera toggle controls.

The system operates in two modes:

- Standalone Mode (main.py / main\_full.py) — OpenCV GUI window with all detection, tracking, visualization, and alerting.
- Client-Server Mode (server.py + React frontend) — Flask REST API backend serving video streams and status data to a React-based web dashboard.

### IMPORTANT

The system is designed as an academic/prototype project. It is not intended for production-grade traffic management deployment without additional hardening, scalability improvements, and regulatory compliance.

## 1.3 Definitions, Acronyms, and Abbreviations

| Term      | Definition                                                                                                              |
|-----------|-------------------------------------------------------------------------------------------------------------------------|
| TCS       | Traffic Congestion System — the name of this project                                                                    |
| YOLO      | You Only Look Once — a family of real-time object detection neural networks                                             |
| YOLO v8   | Version 8 of the YOLO architecture by Ultralytics                                                                       |
| YOLO v8n  | YOLOv8 Nano — the smallest and fastest variant of YOLOv8                                                                |
| ByteTrack | A multi-object tracking algorithm that associates detections across frames using IoU (Intersection over Union) matching |
| BoTSORT   | Another multi-object tracking algorithm (alternative to ByteTrack)                                                      |
| ROI       | Region of Interest — a polygon-defined area within a video frame where vehicle counting is performed                    |
| COCO      | Common Objects in Context — a large-scale object detection dataset; YOLOv8 is pre-trained on COCO                       |
| OpenCV    | Open Source Computer Vision Library (cv2 in Python)                                                                     |
| Flask     | A lightweight Python web framework for building REST APIs                                                               |

|         |                                                                                      |
|---------|--------------------------------------------------------------------------------------|
| CORS    | Cross-Origin Resource Sharing — HTTP header mechanism to allow cross-origin requests |
| MJPE    | Motion JPEG — a video format where each frame is a separately compressed JPEG image  |
| SMTP    | Simple Mail Transfer Protocol — used to send email alerts                            |
| SSL/TLS | Secure Sockets Layer / Transport Layer Security — encryption for SMTP connections    |
| FPS     | Frames Per Second — measure of video processing speed                                |
| SRS     | Software Requirements Specification                                                  |
| GUI     | Graphical User Interface                                                             |
| API     | Application Programming Interface                                                    |
| REST    | Representational State Transfer — architectural style for web APIs                   |
| IoU     | Intersection over Union — metric for overlap between bounding boxes                  |
| BGR     | Blue-Green-Red — OpenCV's default color channel ordering                             |
| Vite    | A modern frontend build tool for fast development                                    |
| TSX     | TypeScript with JSX — React component file format                                    |

## 1.4 References

| #  | Reference                                                           | Description                                                                         |
|----|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1  | IEEE Std 830-1998                                                   | IEEE Recommended Practice for Software Requirements Specifications                  |
| 2  | Ultralytics YOLOv8 Documentation                                    | <a href="https://docs.ultralytics.com/">https://docs.ultralytics.com/</a>           |
| 3  | ByteTrack: Multi-Object Tracking by Associating Every Detection Box | Zhang et al., ECCV 2022                                                             |
| 4  | OpenCV Documentation                                                | <a href="https://docs.opencv.org/">https://docs.opencv.org/</a>                     |
| 5  | Flask Documentation                                                 | <a href="https://flask.palletsprojects.com/">https://flask.palletsprojects.com/</a> |
| 6  | React Documentation                                                 | <a href="https://react.dev/">https://react.dev/</a>                                 |
| 7  | COCO Dataset                                                        | <a href="https://cocodataset.org/">https://cocodataset.org/</a>                     |
| 8  | ipinfo.io API                                                       | <a href="https://ipinfo.io/">https://ipinfo.io/</a> — IP-based geolocation service  |
| 9  | Vite Documentation                                                  | <a href="https://vite.dev/">https://vite.dev/</a>                                   |
| 10 | Matplotlib Documentation                                            | <a href="https://matplotlib.org/">https://matplotlib.org/</a>                       |

## 1.5 Overview

The remainder of this SRS is organized as follows:

- Section 2 provides a high-level overview of the product, its functions, users, and constraints.

- Section 3 describes the system architecture with diagrams.
- Section 4 details all functional and non-functional requirements.
- Section 5 provides an in-depth feature-by-feature specification.
- Sections 6–8 cover data storage, UI design, and API specifications.
- Sections 9–15 cover technology stack, file mapping, testing, deployment, risks, and future work.

## 2. Overall Description

### 2.1 Product Perspective

TCS is a self-contained, standalone system that does not depend on any external traffic management infrastructure. It operates entirely on local hardware (a computer with a webcam) and uses internet connectivity only for:

- Sending email alerts via Gmail SMTP
- Fetching approximate geo-location via the ipinfo.io API

The system is composed of two independently deployable sub-systems:

```
graph TB
    subgraph "Backend (Python)"
        A["Video Capture (OpenCV)"]
        B["Vehicle Detection (YOLOv8n)"]
        C["Multi-Object Tracking (ByteTrack)"]
        D["Congestion Logic Engine"]
        E["Alert System (Email + Notifications)"]
        F["Flask REST API Server"]
        G["MJPEG Video Streamer"]
    end

    subgraph "Frontend (React + TypeScript)"
        H["Web Dashboard"]
        I["Live Video Feed"]
        J["Status Metrics Panel"]
        K["Camera Control UI"]
    end

    A --> B --> C --> D --> E
    D --> F
    A --> G
    F <--> H
    G --> I
    F <--> K
    D --> J
```

### 2.2 Product Functions

The system provides the following major functions:

| #   | Function                | Description                                                                                                                            |
|-----|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| F1  | Vehicle Detection       | Detect cars, motorcycles, buses, and trucks in video frames using YOLOv8n pre-trained on COCO dataset (classes 2, 3, 5, 7)             |
| F2  | Vehicle                 | Assign persistent unique IDs to detected vehicles across frames using ByteTrack algorithm                                              |
| F3  | ROI-Based Counting      | Count vehicles only within a user-defined or auto-calculated Region of Interest polygon                                                |
| F4  | Congestion Detection    | Determine congestion when vehicle count exceeds a threshold for a sustained duration (default: >10 vehicles for >10 seconds)           |
| F5  | Email Alert             | Send email with congestion image attachment, vehicle count, timestamp, and Google Maps location link                                   |
| F6  | Desktop Notification    | Trigger OS-level desktop notification on congestion detection                                                                          |
| F7  | Audible Alert           | Emit terminal beep sound every 30 frames during active congestion                                                                      |
| F8  | Image Capture           | Save timestamped and geo-tagged JPEG images of congestion events                                                                       |
| F9  | Analytics Graph         | Generate and save time-series graphs of vehicle count over time (Matplotlib)                                                           |
| F10 | Geo-Location            | Fetch approximate device coordinates via IP-based geolocation (ipinfo.io)                                                              |
| F11 | Live Dashboard          | Web-based React dashboard showing real-time metrics, camera feed, and controls                                                         |
| F12 | Video                   | Stream MJPEG video from Flask backend to web frontend                                                                                  |
| F13 | Camera Control          | Remote camera on/off toggle via REST API from web dashboard                                                                            |
| F14 | Real-Time Visualization | Overlay bounding boxes, vehicle IDs, status bars, FPS counter, congestion timer, heatmap, and blinking warning banners on video frames |

## 2.3 User Classes and Characteristics

| User Class                    | Description                                                 | Technical Proficiency | Primary Interactions                                             |
|-------------------------------|-------------------------------------------------------------|-----------------------|------------------------------------------------------------------|
| Traffic Monitor Operator      | Personnel responsible for monitoring traffic conditions     | Low to Medium         | View live dashboard, observe alerts, review congestion images    |
| System Administrator          | Person responsible for configuring and deploying the system | High                  | Configure thresholds, manage email settings, start/stop services |
| Traffic Authority / Recipient | Receives email alerts about congestion events               | Low                   | Reads email notifications, clicks Google Maps links              |
| Academic Evaluator            | Reviews the system for academic assessment                  | Medium to High        | Examines code, architecture, functionality                       |

## 2.4 Operating Environment

| Component        | Requirement                                                                     |
|------------------|---------------------------------------------------------------------------------|
| Operating System | macOS, Linux, or Windows (tested on macOS)                                      |
| Python           | Python 3.8 or higher                                                            |
| Node.js          | Node.js 18+ (for React frontend build)                                          |
| GPU (Optional)   | NVIDIA CUDA-compatible GPU for accelerated inference (falls back to CPU)        |
| Webcam           | USB or built-in webcam (accessible as device index 0)                           |
| Internet         | Required for email alerts and geo-location API; not required for core detection |
| Browser          | Modern browser (Chrome, Firefox, Safari, Edge) for web dashboard                |
| RAM              | Minimum 4 GB (8 GB recommended due to PyTorch + YOLOv8 model loading)           |
| Disk Space       | ~500 MB for dependencies + ~6.5 MB for YOLOv8n model weights                    |

## 2.5 Design and Implementation Constraints

1. Single Camera: The current implementation supports a single video source (webcam index 0 or a single video file path). Multi-camera support is not implemented.
2. IP-Based Location: Geo-location is approximate (city-level accuracy) since it uses IP-based detection via ipinfo.io, not GPS hardware.
3. Email Credentials: Sender email credentials (Gmail App Password) are currently hardcoded in the source code. This is a security concern for production use.
4. Frame Skipping: To maintain real-time performance, the system processes only every 3rd frame (frame\_skip = 2) for detection/tracking; intermediate frames reuse previous detection results for visualization.
5. No Database: The system does not use a traditional database. State is held in-memory (Python dictionaries / React state). Persistent artifacts are file-system-based (images, graphs).
6. Single-Threaded Detection: The standalone mode runs detection in the main thread. The server mode runs detection in a background thread to keep the Flask API responsive.

## 2.6 Assumptions and Dependencies

Assumptions:

1. The webcam is accessible at device index 0 and provides at least 640×480 resolution.
2. The deployment machine has internet access for email sending and geo-location lookup.
3. The Gmail account used for sending alerts has App Passwords enabled (2-Factor Authentication must be active on the Gmail account).
4. The target deployment environment has Python 3.8+ and pip available.
5. Vehicles in the video feed are reasonably visible (no extreme occlusion, adequate lighting, standard camera angles).

External Dependencies:

| Dependency | Version | Purpose |
|------------|---------|---------|
|------------|---------|---------|



|               |                |                                                   |
|---------------|----------------|---------------------------------------------------|
| ultralytics   | 8.4.14         | YOLOv8 object detection & tracking framework      |
| opencv-python | 4.13.0.92      | Video capture, frame processing, image I/O        |
| torch         | 2.10.0         | PyTorch deep learning framework (YOLOv8 backend)  |
| torchvision   | 0.25.0         | Vision utilities for PyTorch                      |
| numpy         | 2.4.2          | Numerical array operations                        |
| matplotlib    | 3.10.8         | Graph/chart generation                            |
| flask         | (in server.py) | REST API web server                               |
| flask-cors    | (in server.py) | Cross-origin request handling                     |
| requests      | 2.32.5         | HTTP client for ipinfo.io API                     |
| plyer         | 2.1.0          | Cross-platform desktop notifications              |
| certifi       | 2026.1.4       | SSL certificate bundle for secure email           |
| lapx          | latest         | Linear assignment problem solver for ByteTrack    |
| react         | 19.2.8         | Frontend UI library                               |
| react-dom     | 19.2.8         | React DOM renderer                                |
| lucide-react  | 1.28.0         | Icon library for dashboard UI                     |
| vite          | 8.2.0          | Frontend build tool and dev server                |
| typescript    | 6.0.2          | Type-safe JavaScript for frontend                 |
| concurrently  | 10.0.4         | Run frontend + backend dev servers simultaneously |

## 3. System Architecture

### 3.1 High-Level Architecture Diagram

```

graph LR
    subgraph "Input Layer"
        CAM[" Webcam / Video File"]
    end

    subgraph "Processing Layer (Python Backend)"
        VC["Video Capture\n(OpenCV)"]
        YD["YOLOv8\n\nVehicle Detection"]
        BT["ByteTrack\n\nMulti-Object Tracker"]
        CD["Congestion\n\nDetector"]
        VIZ["Frame\n\nVisualization"]
    end

    subgraph "Alert Layer"
        EA[" Email Alert\n\n(SMTP/SSL)"]
        DN[" Desktop\n\nNotification"]
        IC[" Image\n\nCapture"]
        AG[" Analytics\n\nGraph"]
    end

    subgraph "Service Layer"
        LS[" Location\n\nService (ipinfo.io)"]
        FA["Flask API\n\nServer :5001"]
        VS["MJPEG Video\n\nStream"]
    end

    subgraph "Presentation Layer (React Frontend)"
        DB[" Web Dashboard\n\n(Vite :5173)"]
    end

    CAM --> VC --> YD --> BT --> CD
    CD -->|"congested"| EA & DN & IC & AG
    CD --> VIZ
    VC --> VIZ
    LS --> EA
    LS --> IC
    CD --> FA
    VIZ --> VS
    FA --> DB
    VS --> DB
    DB -->|"toggle camera"| FA

```

## 3.2 Module Decomposition

The system is decomposed into the following modules:

```

graph TD
    TCS["TCS System"]

    TCS --> M1["Vehicle Tracker\n(tracker.py)"]
    TCS --> M2["Congestion Logic\n(congestion_logic.py)"]
    TCS --> M3["Location Service\n(location_service.py)"]
    TCS --> M4["Utility Functions\n(utils.py)"]
    TCS --> M5["Simple Detector\n(detector.py)"]
    TCS --> M6["Main Application\n(main.py / main_full.py)"]
    TCS --> M7["Flask API Server\n(server.py)"]
    TCS --> M8["Web Dashboard\n(frontend/src/App.tsx)"]
    TCS --> M9["Camera Test\n(test_camera.py)"]

    M1 --> |uses| U1["ultralytics.YOLO"]
    M2 --> |uses| U2["time module"]
    M3 --> |uses| U3["requests + ipinfo.io"]
    M4 --> |uses| U4["cv2 + numpy"]
    M6 --> M1 & M2 & M3 & M4
    M7 --> M1 & M2 & M3
    M8 --> |HTTP| M7

```

## Module Descriptions

| Module              | File                | Lines of Code | Responsibility                                                                                          |
|---------------------|---------------------|---------------|---------------------------------------------------------------------------------------------------------|
| Vehicle Tracker     | tracker.py          | 16            | Wraps YOLOv8 model's .track() method; filters for vehicle classes (car=2, motorcycle=3, bus=5, truck=7) |
| Congestion Detector | congestion_logic.py | 28            | Stateful threshold-and-duration-based congestion detection engine                                       |
| Location Service    | location_service.py | 23            | Fetches device GPS coordinates via ipinfo.io API; returns lat, lon, Google Maps link                    |
| Utilities           | utils.py            | 37            | Drawing helpers: draw_text(), draw_roi(), is_inside_roi()                                               |
| Simple              | detector.py         | 20            | Standalone detection-only module (no tracking); detects cars, trucks, buses                             |
| Main (Standard)     | main.py             | 367           | Full standalone application with heatmap overlay, dashboard panels, all alerting                        |
| Main (Full)         | main_full.py        | 257           | Alternative standalone app with explicit ROI polygon filtering                                          |
| Flask Server        | server.py           | 233           | REST API + MJPEG streaming backend for web dashboard                                                    |
| React Dashboard     | App.tsx             | 354           | Web-based monitoring dashboard with live feed and metrics                                               |
| Camera Test         | test_camera.py      | 17            | Simple webcam connectivity test utility                                                                 |

## 3.3 Data Flow Diagram

### Level 0 (Context Diagram)

```
graph LR
    U["User / Operator"] -->|"Views Dashboard\nToggles Camera"| TCS["TCS System"]
    CAM["Webcam"] -->|"Video Frames"| TCS
    TCS -->|"Email Alert"| AUTH["Traffic Authority"]
    TCS -->|"Desktop Notification"| U
    TCS -->|"Congestion Images\n& Analytics Graphs"| FS["File System"]
    IPINFO["ipinfo.io"] -->|"Lat/Lon Coordinates"| TCS
    GMAIL["Gmail SMTP"] -->|"Email Delivery"| AUTH
    TCS -->|"Email via SMTP"| GMAIL
```

## Level 1 (Detailed Data Flow)

```
graph TD
    CAM["Video Source"] -->|"Raw Frame"| P1["1.0 Capture Frame"]
    P1 -->|"Frame (numpy array)"| P2["2.0 Detect Vehicles\n(YOLOv8n)"]
    P2 -->|"Detections\n(bboxes, classes, confidences)"| P3["3.0 Track Vehicles\n(ByteTrack)"]
    P3 -->|"Tracked Vehicles\n(bboxes + persistent IDs)"| P4["4.0 Count in ROI"]
    P4 -->|"vehicle_count (int)"| P5["5.0 Detect Congestion"]

    P5 -->|"is_congested = true"| P6["6.0 Send Email Alert"]
    P5 -->|"is_congested = true"| P7["7.0 Save Image"]
    P5 -->|"is_congested = true"| P8["8.0 Desktop Notify"]
    P5 -->|"is_congested = true"| P9["9.0 Save Graph"]

    P5 -->|"status data"| P10["10.0 Update Global State"]
    P10 -->|"JSON"| API["REST API\n(/api/status)"]

    P1 -->|"Annotated Frame"| P11["11.0 Encode MJPEG"]
    P11 -->|"JPEG stream"| STREAM["/video_feed"]

    API --> DASH["Web Dashboard"]
    STREAM --> DASH

    LOC["ipinfo.io"] -->|"lat, lon"| P6
    LOC -->|"lat, lon"| P7
```

## 4. Specific Requirements

### 4.1 Functional Requirements

#### FR-01: Video Input Acquisition

| Attribu  | Detail                  |
|----------|-------------------------|
| ID       | FR-01                   |
| Title    | Video Input Acquisition |
| Priority | High (Essential)        |

|                |                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Description    | The system shall acquire video frames from either a local webcam (device index 0) or a video file path specified via command-line argument (--source).                              |
| Input          | Webcam device index (integer) or filesystem path to .mp4/.avi video file                                                                                                            |
| Output         | Continuous stream of BGR numpy arrays (video frames)                                                                                                                                |
| Processing     | Uses cv2.VideoCapture() to open the source. Each iteration reads one frame via cap.read(). If the source cannot be opened, the system prints an error message and exits gracefully. |
| Error Handling | If cap.isOpened() returns False, print "Error: Could not open video source" and terminate. If cap.read() returns ret=False, break the main loop.                                    |

## FR-02: Vehicle Detection

| Attribute        | Detail                                                                                                                                                |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID               | FR-02                                                                                                                                                 |
| Title            | Vehicle Detection using YOLOv8                                                                                                                        |
| Priority         | High (Essential)                                                                                                                                      |
| Description      | The system shall detect vehicles in each processed video frame using the YOLOv8n (Nano) deep learning model pre-trained on the COCO dataset.          |
| Detected Classes | Car (COCO class 2), Motorcycle (COCO class 3), Bus (COCO class 5), Truck (COCO class 7)                                                               |
| Output           | For each detected vehicle: bounding box coordinates (x1, y1, x2, y2), confidence score, class ID                                                      |
| Model File       | yolov8n.pt (6.55 MB, stored in project root)                                                                                                          |
| Processing       | The VehicleTracker class loads the YOLO model in __init__() and calls self.model.track() with classes=[2, 3, 5, 7] to filter only vehicle detections. |

## FR-03: Multi-Object Tracking

| Attribute   | Detail                                                                                                                           |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| ID          | FR-03                                                                                                                            |
| Title       | Multi-Object Vehicle Tracking                                                                                                    |
| Priority    | High (Essential)                                                                                                                 |
| Description | The system shall assign persistent unique integer IDs to each detected vehicle and maintain these IDs across consecutive frames. |
| Algorithm   | ByteTrack (configured via bytetrack.yaml). Alternative: BoTSORT (configurable).                                                  |
| Parameters  | tracker="bytetrack.yaml", persist=True (maintains track state across calls), verbose=False                                       |

|             |                                                                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Output      | For each tracked vehicle: bounding box + unique track ID (integer)                                                                                                 |
| Persistence | <code>persist=True</code> ensures the tracker maintains internal state between consecutive calls to <code>.track()</code> , enabling ID persistence across frames. |

## FR-04: Region of Interest Filtering

| Attribute        | Detail                                                                                                                                                                                                                              |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID               | FR-04                                                                                                                                                                                                                               |
| Title            | ROI-Based Vehicle Counting                                                                                                                                                                                                          |
| Priority         | Medium                                                                                                                                                                                                                              |
| Description      | In <code>main_full.py</code> mode, the system shall define a polygon ROI covering the central 80% of the frame (from 10% to 90% width, 20% to 100% height) and count only vehicles whose bottom-center point falls inside this ROI. |
| ROI Definition   | <code>roi_points = [(0.1w, 1.0h), (0.1w, 0.2h), (0.9w, 0.2h), (0.9w, 1.0h)]</code>                                                                                                                                                  |
| Point-in-Polygon | Uses <code>cv2.pointPolygonTest()</code> — returns $\geq 0$ if the point is inside or on the boundary.                                                                                                                              |
| Visualization    | ROI polygon is drawn on every frame using <code>cv2.polylines()</code> with yellow color (255, 255, 0).                                                                                                                             |

## FR-05: Congestion Detection

| Attribute            | Detail                                                                                                                                                                                                                            |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID                   | FR-05                                                                                                                                                                                                                             |
| Title                | Threshold-Based Congestion Detection                                                                                                                                                                                              |
| Priority             | High (Essential)                                                                                                                                                                                                                  |
| Description          | The system shall detect congestion when the number of tracked vehicles exceeds a configurable threshold for a sustained configurable duration.                                                                                    |
| Default Threshold    | 10 vehicles (standalone mode), 6 vehicles ( <code>main_full.py</code> mode)                                                                                                                                                       |
| Default Duration     | 10 seconds (standalone mode), 6 seconds ( <code>main_full.py</code> mode)                                                                                                                                                         |
| Algorithm            | If <code>vehicle_count &gt; threshold</code> : start timer. If timer exceeds duration: set <code>congestion = True</code> . If count drops below threshold: reset timer and congestion state, clear <code>alert_sent</code> flag. |
| State Variables      | <code>threshold</code> , <code>duration</code> , <code>elapsed_time</code> , <code>start_time</code> , <code>congestion (bool)</code> , <code>alert_sent (bool)</code>                                                            |
| Alert De-duplication | <code>alert_sent</code> flag ensures only one alert is sent per congestion event. Flag resets when traffic normalizes.                                                                                                            |

## FR-06: Email Alert Notification

| Attribute      | Detail                                                                                                                                                                                                                       |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID             | FR-06                                                                                                                                                                                                                        |
| Title          | Automated Email Alert                                                                                                                                                                                                        |
| Priority       | High                                                                                                                                                                                                                         |
| Description    | Upon congestion detection (first occurrence per event), the system shall send an email alert containing the vehicle count, timestamp, geo-coordinates, Google Maps link, and a JPEG image attachment of the congested scene. |
| Protocol       | SMTP over SSL (port 465, smtp.gmail.com)                                                                                                                                                                                     |
| Authentication | Gmail App Password                                                                                                                                                                                                           |
| Email Content  | Subject: " Traffic Congestion Alert - TCS"; Body: vehicle count, time, lat/lon, Google Maps link; Attachment: congestion JPEG image                                                                                          |
| Threading      | In server mode, email sending runs in a separate daemon thread to prevent blocking the video processing loop.                                                                                                                |

## FR-07: Desktop Notification

| Attribute        | Detail                                                                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID               | FR-07                                                                                                                                                     |
| Title            | OS-Level Desktop Notification                                                                                                                             |
| Priority         | Low                                                                                                                                                       |
| Description      | The system shall trigger a native desktop notification with title "Traffic Congestion Alert" and message "Traffic Jam Detected!" using the plyer library. |
| Timeout          | 5 seconds                                                                                                                                                 |
| Platform Support | macOS, Linux, Windows (via plyer abstraction). Failures are caught and logged without crashing the application.                                           |

## FR-08: Congestion Image Capture

| Attribute | Detail                           |
|-----------|----------------------------------|
| ID        | FR-08                            |
| Title     | Congestion Evidence Image Saving |
| Priority  | Medium                           |

|                  |                                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Description      | When congestion is detected, the system shall save the current frame as a JPEG file with timestamp and geo-coordinates overlaid on the image. |
| Output Directory | congestion_images/                                                                                                                            |
| Filename Format  | congestion_YYYY-MM-DD_HH-MM-SS.jpg                                                                                                            |
| Overlaid Data    | Timestamp text at (20, 50), Lat/Lon text at (20, 80) — both in red color                                                                      |

## FR-09: Analytics Graph Generation

| Attribute        | Detail                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------|
| ID               | FR-09                                                                                                                 |
| Title            | Vehicle Count Time-Series Graph                                                                                       |
| Priority         | Medium                                                                                                                |
| Description      | The system shall generate and save a time-series line graph of vehicle counts over processed frames using Matplotlib. |
| Output Directory | analytics_graphs/                                                                                                     |
| Filename Format  | vehicle_graph_YYYY-MM-DD_HH-MM-SS.png                                                                                 |
| Graph Details    | Title: "Vehicle Count Over Time"; X-axis: "Frames"; Y-axis: "Vehicle Count"; Grid enabled; Figure size: 10×5 inches   |
| Trigger          | Generated on congestion detection AND on application exit (standalone mode)                                           |

## FR-10: Live Video Streaming

| Attribute        | Detail                                                                                                        |
|------------------|---------------------------------------------------------------------------------------------------------------|
| ID               | FR-10                                                                                                         |
| Title            | MJPEG Video Stream                                                                                            |
| Priority         | High (for server mode)                                                                                        |
| Description      | The Flask backend shall stream annotated video frames as a continuous MJPEG stream accessible at /video_feed. |
| Content Type     | multipart/x-mixed-replace; boundary=frame                                                                     |
| Frame Rate       | ~20 FPS emission rate (time.sleep(0.05))                                                                      |
| Offline Handling | When camera is off, a black frame with "CAMERA OFFLINE" text is streamed.                                     |

## FR-11: REST API Status Endpoint



| Attribute       | Detail                                                                                                                                                |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID              | FR-11                                                                                                                                                 |
| Title           | System Status API                                                                                                                                     |
| Priority        | High (for server mode)                                                                                                                                |
| Description     | The Flask backend shall expose a GET /api/status endpoint returning current system state as JSON.                                                     |
| Response Fields | vehicle_count (int), is_congested (bool), latitude (float), longitude (float), map_link (string), camera_active (bool), last_alert_time (float/epoch) |

## FR-12: Camera Toggle API

| Attribute       | Detail                                                                                                                                                        |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID              | FR-12                                                                                                                                                         |
| Title           | Camera On/Off Remote Control                                                                                                                                  |
| Priority        | Medium                                                                                                                                                        |
| Description     | The Flask backend shall expose a POST /api/camera/toggle endpoint that accepts {"active": true/false} to start/stop the camera feed and detection processing. |
| Behavior on Off | Releases the VideoCapture object, resets vehicle count to 0 and congestion state to false.                                                                    |

## FR-13: Real-Time Web Dashboard

| Attribute          | Detail                                                                                                                                                          |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID                 | FR-13                                                                                                                                                           |
| Title              | React-Based Monitoring Dashboard                                                                                                                                |
| Priority           | High (for server mode)                                                                                                                                          |
| Description        | The frontend shall display a real-time dashboard with live video feed, current traffic status, vehicle count, location coordinates, and camera toggle controls. |
| Polling Interval   | 1 second (setInterval(fetchStatus, 1000))                                                                                                                       |
| Toast Notification | When a new email alert is sent (detected via last_alert_time change), a toast popup appears for 5 seconds with "Report Sent Successfully" message.              |

## FR-14: Frame-Level Visualization

| Attribute | Detail |
|-----------|--------|
|-----------|--------|

|                             |                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID                          | FR-14                                                                                                                                                                                                                                                                                                                                |
| Title                       | Real-Time Video Overlay Visualization                                                                                                                                                                                                                                                                                                |
| Priori                      | Medium                                                                                                                                                                                                                                                                                                                               |
| Desc<br>ription             | Each displayed frame shall include visual overlays for operational awareness.                                                                                                                                                                                                                                                        |
| Overl<br>ay<br>Elem<br>ents | Bounding boxes (green=normal, red=alert), Vehicle ID labels, Header bar with title + status, Bottom panel with vehicle count + threshold info + live clock, FPS counter, Congestion progress bar, Blinking "TRAFFIC CONGESTION DETECTED" banner, Heatmap overlay on vehicle areas (standalone mode), ROI polygon (main_full.py mode) |

## 4.2 Non-Functional Requirements

### NFR-01: Performance

| Attribute         | Detail                                                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------|
| ID                | NFR-01                                                                                                                        |
| Description       | The system shall process video in near real-time, maintaining a minimum of 10 FPS on a modern laptop with CPU-only inference. |
| Frame<br>Skipping | Processing every 3rd frame (frame_skip = 2) to maintain performance. Intermediate frames reuse previous detection results.    |
| Model Size        | YOLOv8n (Nano) chosen as the smallest variant for maximum inference speed.                                                    |

### NFR-02: Reliability

| Attribute         | Detail                                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| ID                | NFR-02                                                                                                          |
| Description       | The system shall gracefully handle external service failures (email, location API) without crashing.            |
| Error<br>Handling | All network operations (email, HTTP requests) are wrapped in try/except blocks with descriptive error messages. |

### NFR-03: Usability

| Attrib<br>ute   | Detail                                                                                                                                              |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| ID              | NFR-03                                                                                                                                              |
| Descri<br>ption | The web dashboard shall be intuitive and require no technical training to operate. The standalone OpenCV GUI shall display clear status indicators. |

Contro Single key ('q') to quit standalone mode. Single button to toggle camera in web mode.

Is

## NFR-04: Maintainability

| Attribute    | Detail                                                                                                                          |
|--------------|---------------------------------------------------------------------------------------------------------------------------------|
| ID           | NFR-04                                                                                                                          |
| Description  | The codebase shall follow modular design with separation of concerns. Each Python module handles a single                       |
| Module       | tracker.py (detection), congestion_logic.py (analysis), location_service.py (geo), utils.py (drawing), server.py (API), main.py |
| Independence | (orchestration)                                                                                                                 |
| e            |                                                                                                                                 |

## NFR-05: Portability

| Attribute   | Detail                                                                                                                                             |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| ID          | NFR-05                                                                                                                                             |
| Description | The system shall run on macOS, Linux, and Windows without code modification (excluding platform-specific notification behavior handled by pleyer). |

## NFR-06: Scalability Constraints

| Attribute   | Detail                                                                                                                                                                      |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID          | NFR-06                                                                                                                                                                      |
| Description | The current system supports a single camera feed. Scaling to multiple cameras would require architectural changes (multi-threaded processing, per-camera state management). |

## NFR-07: Security

| Attribute   | Detail                                                                                                                                                                                                        |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID          | NFR-07                                                                                                                                                                                                        |
| Description | Email credentials are currently hardcoded. For production, credentials should be managed via environment variables or a secrets manager. SSL/TLS is used for SMTP connections via certifi certificate bundle. |

## 4.3 External Interface Requirements

### 4.3.1 User Interfaces

| Interface     | Type          | Description                                                                       |
|---------------|---------------|-----------------------------------------------------------------------------------|
| OpenCV Window | Desktop GUI   | Real-time video display with overlays (1200×800, resizable)                       |
| Web Dashboard | Browser-based | React SPA accessible at <a href="http://localhost:5173">http://localhost:5173</a> |

### 4.3.2 Hardware Interfaces

| Interface | Description                                                  |
|-----------|--------------------------------------------------------------|
| Webcam    | USB or built-in camera accessed via OpenCV's VideoCapture(0) |
| Display   | Monitor for viewing OpenCV window or browser dashboard       |

### 4.3.3 Software Interfaces

| Interface      | Protocol                 | Description                                    |
|----------------|--------------------------|------------------------------------------------|
| Gmail SMTP     | SMTP over SSL (port 465) | Sends email alerts with image attachments      |
| ipinfo.io API  | HTTPS GET                | Returns JSON with IP-based geolocation data    |
| Flask REST API | HTTP (port 5001)         | Backend API for status data and camera control |
| MJPEG Stream   | HTTP multipart           | Live video stream endpoint                     |

### 4.3.4 Communication Interfaces

| Interface           | Details                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------|
| Frontend ↔ Backend  | HTTP/1.1 over localhost (port 5001). JSON request/response for API. MJPEG for video.           |
| Backend ↔ Gmail     | SMTP_SSL connection to smtp.gmail.com:465                                                      |
| Backend ↔ ipinfo.io | HTTPS GET to <a href="https://ipinfo.io/json">https://ipinfo.io/json</a> with 5-second timeout |

## 5. System Features (Detailed)

### 5.1 Vehicle Detection & Classification

Purpose: Identify and locate vehicles in each video frame.

Technical Implementation:

- Model: YOLOv8n (Nano) — smallest variant of the YOLOv8 architecture
- Model File: yolov8n.pt (6,549,796 bytes)
- Framework: Ultralytics Python SDK (from `ultralytics import YOLO`)

- Input: Single BGR frame (numpy array of shape H×W×3)
- COCO Class Filtering: classes=[2, 3, 5, 7]
  - Class 2: Car
  - Class 3: Motorcycle
  - Class 5: Bus
  - Class 7: Truck
- Output per detection: Bounding box (x1, y1, x2, y2), confidence score, class ID

Code Reference: tracker.py — VehicleTracker class, line 14:

```
results = self.model.track(frame, tracker=tracker, persist=persist, verbose=verbose, classes=[2, 3, 5, ...
```

## 5.2 Multi-Object Tracking

Purpose: Maintain persistent identity for each vehicle across frames.

Technical Implementation:

- Algorithm: ByteTrack (default), with BoTSORT as alternative
- Configuration: tracker="bytetrack.yaml" (Ultralytics built-in config)
- Persistence: persist=True maintains the tracker's internal state across consecutive .track() calls
- Track ID Assignment: Each vehicle receives a unique integer ID that persists as long as the vehicle remains visible
- Output: results.bboxes.id — tensor of track IDs corresponding to each detection

ByteTrack vs BoTSORT:

| Feature            | ByteTrack      | BoTSORT                    |
|--------------------|----------------|----------------------------|
| Speed              | Faster         | Slightly slower            |
| Re-identification  | IoU-based only | IoU + appearance features  |
| Occlusion handling | Good           | Better                     |
| Dependency         | lapx           | lapx + optional ReID model |

## 5.3 Region of Interest (ROI) Filtering

Purpose: Count only vehicles within a defined area to avoid false positives from peripheral traffic.

Available in: main\_full.py only

ROI Polygon Definition:

```
roi_points = np.array([
    [int(w*0.1), int(h*1.0)], # bottom-left
    [int(w*0.1), int(h*0.2)], # top-left
    [int(w*0.9), int(h*0.2)], # top-right
    [int(w*0.9), int(h*1.0)]  # bottom-right
], np.int32)
```

Point-in-Polygon Test:

```
# Center-bottom point of bounding box
center_x = int((x1 + x2) / 2)
center_y = int(y2)
result = cv2.pointPolygonTest(roi_points, (center_x, center_y), False)
# result >= 0 means inside or on boundary
```

## 5.4 Congestion Detection Engine

Purpose: Determine if traffic is congested based on sustained high vehicle counts.

State Machine:

```
stateDiagram-v2
    [*] --> Normal
    Normal --> Timing: vehicle_count > threshold
    Timing --> Normal: vehicle_count ≤ threshold
    Timing --> Congested: elapsed_time > duration
    Congested --> Normal: vehicle_count ≤ threshold
    Congested --> AlertSent: send_alert()
    AlertSent --> Normal: vehicle_count ≤ threshold
```

Configuration Parameters:

| Parameter | Default (main.py) | Default (main_full.py) | Description                                |
|-----------|-------------------|------------------------|--------------------------------------------|
| threshold | 10                | 6                      | Minimum vehicles to start congestion timer |
| duration  | 10 sec            | 6 sec                  | How long threshold must be exceeded        |

Code Reference: `congestion_logic.py` — `CongestionDetector` class

## 5.5 Email Alert System

Purpose: Notify traffic authorities about congestion events with evidence.

Email Composition:

| Field   | Value                          |
|---------|--------------------------------|
| Subject | Traffic Congestion Alert - TCS |

|            |                                                                        |
|------------|------------------------------------------------------------------------|
| From       | harshrajs1k@gmail.com                                                  |
| To         | Configured receiver email                                              |
| Body       | Vehicle count, timestamp, lat/lon coordinates, Google Maps link        |
| Attachment | JPEG image of congestion scene with overlaid timestamp and coordinates |

Security: Uses `ssl.create_default_context(cafile=certifi.where())` for SSL certificate verification.

Threading (Server Mode): Email is sent in a daemon thread to avoid blocking the video stream:

```
threading.Thread(target=send_email_alert, args=..., daemon=True).start()
```

## 5.6 Desktop Notification System

Purpose: Alert the local operator via OS-level notification.

Implementation:

```
from plyer import notification
notification.notify(
    title="Traffic Congestion Alert",
    message="Traffic Jam Detected!",
    timeout=5
)
```

Error Tolerance: Wrapped in try/except to handle platforms where notifications are not supported.

## 5.7 Geo-Location Service

Purpose: Determine approximate device location for embedding in alerts.

API Used: <https://ipinfo.io/json>

Response Parsing:

```
data = response.json()
coords = data.get("loc", "") # format: "lat,long"
latitude, longitude = coords.split(",")
map_link = f"https://maps.google.com/?q={latitude},{longitude}"
```

Limitations:

- Accuracy: City-level (IP-based, not GPS)
- Timeout: 5 seconds
- Fallback: Returns "Unknown", "Unknown", "Unavailable" on failure

Code Reference: `location_service.py`

## 5.8 Congestion Image Capture

Purpose: Save photographic evidence of congestion events.

Process:

1. Overlay timestamp text on frame at position (20, 50)
2. Overlay latitude/longitude text at position (20, 80)
3. Save as JPEG to congestion\_images/ directory
4. Return filename for email attachment

## 5.9 Analytics Graph Generation

Purpose: Provide historical vehicle count trends.

Implementation:

- Uses Matplotlib to plot vehicle\_history list
- Rolling window of last 100 data points (oldest entries are popped)
- Graph is saved as PNG (10×5 inch figure)
- Generated both during congestion events and on application exit

## 5.10 Real-Time Dashboard (Web Frontend)

Purpose: Provide a browser-based monitoring interface.

Components:

1. TopNav — Navigation bar with "TCS Dashboard" branding and menu links
2. MetricCard — Reusable card showing title, value, subtitle, and icon
3. CameraFeed — Live MJPEG video viewer with camera toggle and status badge
4. Toast Notification — Popup when email alert is sent, visible for 5 seconds

State Management:

```
const [vehicleCount, setVehicleCount] = useState(0);
const [isCongested, setIsCongested] = useState(false);
const [isCameraActive, setIsCameraActive] = useState(false);
const [location, setLocation] = useState({ lat: 23.0225, lon: 72.5714 });
const [showPopup, setShowPopup] = useState(false);
const lastAlertTimeRef = useRef(0);
```

## 5.11 Live Video Streaming

Purpose: Deliver real-time annotated video to the web frontend.

Technical Details:

- Format: MJPEG (Motion JPEG)



- Content-Type: multipart/x-mixed-replace; boundary=frame
- Frame encoding: cv2.imencode('.jpg', frame)
- Frame emission rate: ~20 FPS (time.sleep(0.05))
- Offline frame: 640×480 black image with "CAMERA OFFLINE" text

## 5.12 Camera Control API

Purpose: Allow remote on/off control of the camera feed.

Behavior:

- Turn On: Opens cv2.VideoCapture(0), starts detection loop
- Turn Off: Releases the capture object, resets metrics to zero, streams offline frame
- UI: Toggle button in CameraFeed component with optimistic UI update

## 6. Database / Data Storage Design

The system uses a file-system-based and in-memory storage approach. There is no traditional relational or NoSQL database.

### In-Memory Storage

| Store                       | Type           | Contents                                                                        | Scope                         |
|-----------------------------|----------------|---------------------------------------------------------------------------------|-------------------------------|
| global_state<br>(server.py) | Python dict    | vehicle_count, is_congested, lat, lon, map_link, camera_active, last_alert_time | Flask server process lifetime |
| vehicle_history             | Python list    | Last 100 vehicle count values                                                   | Application session           |
| current_vehicles            | Python list    | Current frame's vehicle bounding boxes + track IDs                              | Updated each processed frame  |
| React state                 | useState hooks | vehicleCount, isCongested, isCameraActive, location, showPopup                  | Browser session               |

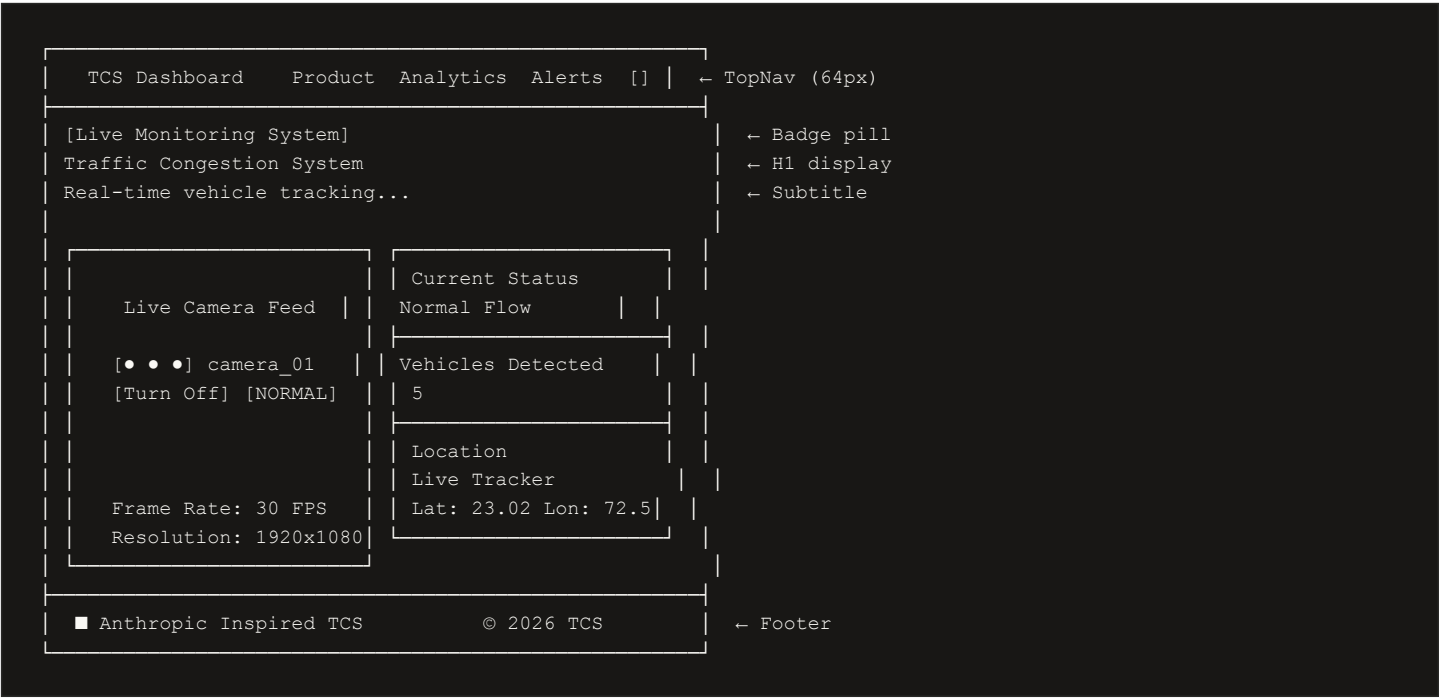
### File-System Storage

| Directory          | File Type | Naming Convention                     | Retention                        |
|--------------------|-----------|---------------------------------------|----------------------------------|
| congestion_images/ | JPEG      | congestion_YYYY-MM-DD_HH-MM-SS.jpg    | Persistent until manual deletion |
| analytics_graphs/  | PNG       | vehicle_graph_YYYY-MM-DD_HH-MM-SS.png | Persistent until manual deletion |

# 7. User Interface Design

## 7.1 Dashboard Layout

The web dashboard follows a two-column layout:



## 7.2 UI Components

| Component            | Description             | Visual Treatment                                                                     |
|----------------------|-------------------------|--------------------------------------------------------------------------------------|
| TopNav               | Fixed navigation bar    | Cream background, hairline border-bottom, 64px height                                |
| MetricCard           | Status/metric display   | Cream card background (#efe9de), 12px border-radius, 32px padding                    |
| CameraFeed           | Live video viewer       | Dark surface background (#181715), macOS-style window dots, monospace filename       |
| Toast Popup          | Email sent notification | Fixed position, slides down from top, cream background with shadow, auto-hides in 5s |
| Badge (Success)      | "NORMAL" status         | Green (#5db872), pill shape, white text                                              |
| Badge (Alert)        | "CONGESTED" status      | Red (#c64545), pill shape, white text                                                |
| Camera Toggle Button | Power on/off            | Dark elevated when on, red when off, Power icon + text label                         |
| Alert Card (Coral)   | Congestion alert info   | Coral background (#cc785c), appears only during congestion                           |

## 7.3 Design System & Theming

The UI follows a warm, editorial design system inspired by Anthropic's Claude interface:

Color Palette:

| Token          | Hex     | Usage                                     |
|----------------|---------|-------------------------------------------|
| --primary      | #cc785c | Coral — CTA buttons, alert cards          |
| --canvas       | #faf9f5 | Warm cream page background                |
| --ink          | #141413 | Primary text color                        |
| --surface-card | #efe9de | Metric card backgrounds                   |
| --surface-dark | #181715 | Camera feed card, footer                  |
| --success      | #5db872 | Normal status badge                       |
| --error        | #c64545 | Alert status badge, congestion indicators |
| --warning      | #d4a017 | Warning states                            |

Typography:

| Element               | Font                         | Size    | Weight |
|-----------------------|------------------------------|---------|--------|
| Display headings (H1) | Times New Roman / Copernicus | 64px    | 400    |
| Body text             | Inter / StyreneB             | 16px    | 400    |
| Code / terminal       | JetBrains Mono               | 14px    | 400    |
| Buttons               | Inter                        | 14px    | 500    |
| Captions / badges     | Inter                        | 12-13px | 500    |

## 8. API Specification

### Endpoint Summary

| Metho | Endpoint           | Description             | Request Body     | Response                  |
|-------|--------------------|-------------------------|------------------|---------------------------|
| GET   | /video_feed        | Live MJPEG video stream | —                | multipart/x-mixed-replace |
| GET   | /api/status        | Current system state    | —                | JSON object               |
| POST  | /api/camera/toggle | Toggle camera on/off    | {"active": bool} | JSON object               |

## GET /api/status

Response (200 OK):

```
{
  "vehicle_count": 7,
  "is_congested": false,
  "latitude": 23.0225,
  "longitude": 72.5714,
  "map_link": "https://maps.google.com/?q=23.0225,72.5714",
  "camera_active": true,
  "last_alert_time": 1723100000.123
}
```

## POST /api/camera/toggle

Request:

```
{
  "active": true
}
```

Response (200 OK):

```
{
  "success": true,
  "camera_active": true
}
```

Response (400 Bad Request):

```
{
  "success": false
}
```

## GET /video\_feed

Response: Continuous MJPEG stream

```
Content-Type: multipart/x-mixed-replace; boundary=frame

--frame
Content-Type: image/jpeg

<JPEG binary data>
--frame
...
```

## 9. Technology Stack

```
graph TB
    subgraph "Frontend"
        R["React 19.2"]
        TS["TypeScript 6.0"]
        V["Vite 8.2"]
        LR["Lucide React Icons"]
        CSS["Vanilla CSS\n(Custom Design System)"]
    end

    subgraph "Backend"
        PY["Python 3.x"]
        FL["Flask + Flask-CORS"]
        CV["OpenCV 4.13"]
        UL["Ultralytics 8.4\n(YOLOv8)"]
        PT["PyTorch 2.10"]
        MP["Matplotlib 3.10"]
        PL["Plyer 2.1"]
    end

    subgraph "AI / ML"
        Y8["YOLOv8n Model\n(COCO Pre-trained)"]
        BT["ByteTrack Algorithm"]
    end

    subgraph "External Services"
        GM["Gmail SMTP\n(SSL/TLS)"]
        IP["ipinfo.io\n(Geo-location)"]
    end

    subgraph "Infrastructure"
        NP["NumPy 2.4"]
        RQ["Requests 2.32"]
        CR["Certifi\n(SSL Certs)"]
    end

    R --> V
    TS --> V
    FL --> PY
    UL --> PT
    UL --> Y8
    Y8 --> BT
    FL --> CV
```

# 10. Module-Wise File Mapping

## Backend (Python)

| File                | Module             | Size    | LOC | Dependencies                                       |
|---------------------|--------------------|---------|-----|----------------------------------------------------|
| tracker.py          | Vehicle Tracker    | 616 B   | 16  | ultralytics                                        |
| congestion_logic.py | Congestion Engine  | 790 B   | 28  | time                                               |
| location_service.py | Geo-Location       | 650 B   | 23  | requests                                           |
| utils.py            | Drawing Utilities  | 1,390 B | 37  | cv2, numpy                                         |
| detector.py         | Simple Detector    | 451 B   | 20  | ultralytics                                        |
| main.py             | Standalone App     | 10,772  | 367 | All modules                                        |
| main_full.py        | ROI Standalone App | 9,042 B | 257 | All modules                                        |
| server.py           | Flask API Server   | 7,792 B | 233 | flask, tracker, congestion_logic, location_service |
| test_camera.py      | Camera Test        | 263 B   | 17  | cv2                                                |

## Frontend (React + TypeScript)

| File      | Component       | Size     | LOC | Dependencies        |
|-----------|-----------------|----------|-----|---------------------|
| App.tsx   | Main Dashboard  | 13,917 B | 354 | react, lucide-react |
| index.css | Design System   | 6,155 B  | 310 | —                   |
| main.tsx  | App Entry Point | 230 B    | ~10 | react-dom           |

## Configuration Files

| File                    | Purpose                              |
|-------------------------|--------------------------------------|
| requirements.txt        | Python dependency list (38 packages) |
| package.json (root)     | Monorepo root with dev script        |
| package.json (frontend) | Frontend dependencies and scripts    |
| vite.config.ts          | Vite build configuration             |
| tsconfig.json           | TypeScript configuration             |
| .gitignore              | Git ignore rules                     |

yolov8n.pt

Pre-trained YOLOv8 Nano model weights (6.55 MB)

# 11. Testing Strategy

## 11.1 Unit Testing

| Module             | Test Cases                                                                                                                                                                                                     |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CongestionDetector | Test threshold boundary (count = threshold, count = threshold+1). Test duration timer (ensure congestion not triggered before duration). Test reset behavior when count drops. Test alert_sent flag lifecycle. |
| VehicleTracker     | Test model loading. Test that .track() returns valid results object. Test class filtering (only classes 2,3,5,7 returned).                                                                                     |
| LocationService    | Test successful API response parsing. Test network timeout handling. Test malformed response handling.                                                                                                         |
| utils              | Test is_inside_roi() with points inside, outside, and on boundary of polygon. Test draw_text() and draw_roi() do not crash on edge cases.                                                                      |

## 11.2 Integration Testing

| Test Scenario        | Description                                                                                                                      |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| End-to-End Detection | Feed a video with known vehicle count; verify detected count matches expected (within tolerance).                                |
| Congestion Trigger   | Feed a video with >10 vehicles for >10 seconds; verify congestion flag activates and email is sent.                              |
| API Roundtrip        | Start Flask server, call /api/status, verify JSON response schema. Toggle camera via /api/camera/toggle and verify state change. |
| Video Stream         | Start server, connect to /video_feed, verify continuous JPEG frames are received.                                                |

## 11.3 Camera Connectivity Test

The project includes test\_camera.py as a dedicated camera test:

```
cap = cv2.VideoCapture(0)
# Opens webcam and displays raw feed
# Press 'q' to quit
```

## 11.4 Manual Testing Checklist

- ☐ Webcam feed displays in OpenCV window
- ☐ Vehicles are detected with bounding boxes
- ☐ Track IDs persist across frames
- ☐ Congestion triggers after threshold + duration
- ☐ Email is received with correct content and image attachment
- ☐ Desktop notification appears
- ☐ Congestion image saved to congestion\_images/
- ☐ Analytics graph saved to analytics\_graphs/
- ☐ Web dashboard displays live video feed
- ☐ Vehicle count updates in real-time on dashboard
- ☐ Camera toggle works from web UI
- ☐ Toast notification appears when email is sent
- ☐ System recovers when camera is disconnected and reconnected

## 12. Deployment & Installation

### 12.1 Prerequisites

1. Python 3.8+ installed
2. Node.js 18+ and npm installed
3. Git installed
4. Webcam connected and accessible

### 12.2 Installation Steps



```
# 1. Clone the repository
git clone https://github.com/justHarshraj/TCS_Traffic_Congestion_System.git
cd TCS_Traffic_Congestion_System

# 2. Create and activate Python virtual environment
python3 -m venv venv
source venv/bin/activate # Mac/Linux
# venv\Scripts\activate # Windows

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Install frontend dependencies
cd frontend
npm install
cd ..
```

## 12.3 Running the Application

### Standalone Mode (OpenCV GUI)

```
# Webcam
python main.py

# Video file
python main.py --source path/to/video.mp4

# With ROI filtering
python main_full.py --source 0
```

### Client-Server Mode (Web Dashboard)

```
# Start both frontend and backend simultaneously
cd frontend
npm run dev

# This runs concurrently:
# - Vite dev server on http://localhost:5173
# - Flask API server on http://localhost:5001
```

### Camera Test

```
python test_camera.py
```

## 12.4 Configuration

| Setting                 | Location                                           | Default               | How to Change                |                        |
|-------------------------|----------------------------------------------------|-----------------------|------------------------------|------------------------|
| Vehicle count threshold | congestion_logic.py constructor / main.py line 140 | 10                    | Change                       | threshold=10 parameter |
| Congestion duration     | congestion_logic.py constructor / main.py line 140 | 10 sec                | Change                       | duration=10            |
| Email sender            | main.py / server.py                                | harshrajs1k@gmail.com | Edit sender_email variable   |                        |
| Email receiver          | main.py / server.py                                | (configured)          | Edit receiver_email variable |                        |
| YOLO model              | tracker.py line 4                                  | yolov8n.pt            | Change                       | model_path             |
| Video source            | CLI argument                                       | 0 (webcam)            | --source <path>              |                        |
| API server port         | server.py line 232                                 | 5001                  | Change port=5001             |                        |
| Frame skip rate         | main.py line 143                                   | 2 (every 3rd frame)   | Change frame_skip value      |                        |

## 13. Risk Analysis

| #   | Risk                                                     | Prob<br>abilit | Impa<br>ct | Mitigation                                                                                  |
|-----|----------------------------------------------------------|----------------|------------|---------------------------------------------------------------------------------------------|
| R1  | Camera not accessible (driver issues, permissions)       | Medi<br>um     | High       | Test with test_camera.py first; verify camera permissions in OS settings                    |
| R2  | Poor detection accuracy in low-light or adverse weather  | Medi<br>um     | Medi<br>um | Use a more accurate model (yolov8s or yolov8m); add pre-processing (histogram equalization) |
| R3  | Email delivery failure (network, SMTP block)             | Medi<br>um     | Medi<br>um | Retry mechanism; log failed alerts to file; support multiple notification channels          |
| R4  | High CPU/memory usage causing lag                        | Medi<br>um     | Medi<br>um | Frame skipping already implemented; reduce input resolution; use GPU if available           |
| R5  | Hardcoded credentials exposed in version control         | High           | High       | Move to environment variables or .env file; add .env to .gitignore                          |
| R6  | Single point of failure (one process, one camera)        | Low            | Medi<br>um | Future: containerized microservices with health checks                                      |
| R7  | ipinfo.io API rate limiting or downtime                  | Low            | Low        | Cache location on startup; provide manual fallback coordinates                              |
| R8  | False positive congestion alerts (parked vehicles, etc.) | Medi<br>um     | Medi<br>um | Tune threshold/duration; add motion-based filtering; use ROI mode                           |
| R9  | CORS issues between frontend and backend                 | Low            | Low        | flask_cors with CORS(app) already configured for all origins                                |
| R10 | Model file (yolov8n.pt) missing or corrupted             | Low            | High       | Ultralytics auto-downloads model if missing; add checksum verification                      |

## 14. Future Enhancements

| #  | Enhancement                      | Description                                                                                          | Priority |
|----|----------------------------------|------------------------------------------------------------------------------------------------------|----------|
| 1  | Multi-Camera Support             | Process feeds from multiple cameras simultaneously with per-camera congestion analysis               | High     |
| 2  | Database Integration             | Store congestion events, vehicle counts, and alerts in PostgreSQL or MongoDB for historical analysis | High     |
| 3  | GPU Acceleration                 | Auto-detect CUDA-compatible GPU and use it for inference to boost FPS                                | Medium   |
| 4  | Environment Variables            | Move all credentials and configuration to .env file with python-dotenv                               | High     |
| 5  | Authentication                   | Add user login/authentication for the web dashboard                                                  | Medium   |
| 6  | Historical Dashboard             | Analytics Add graphs and charts in the web UI showing congestion trends over days/weeks              | High     |
| 7  | Speed Estimation                 | Calculate vehicle speeds using consecutive frame positions and known camera calibration              | Medium   |
| 8  | License Plate Recognition        | Integrate ANPR (Automatic Number Plate Recognition) for vehicle identification                       | Low      |
| 9  | Mobile App                       | React Native or Flutter mobile app for monitoring on the go                                          | Medium   |
| 10 | WebSocket Communication          | Replace HTTP polling with WebSockets for lower-latency real-time updates                             | Medium   |
| 11 | Configurable ROI via UI          | Allow users to draw/edit the ROI polygon interactively from the web dashboard                        | Medium   |
| 12 | Cloud Deployment                 | Containerize with Docker and deploy to AWS/GCP/Azure with auto-scaling                               | Low      |
| 13 | Vehicle Classification Analytics | Break down counts by vehicle type (cars vs trucks vs buses)                                          | Medium   |
| 14 | Night Vision Mode                | Pre-processing pipeline for infrared or low-light cameras                                            | Low      |
| 15 | SMS/WhatsApp Alerts              | Add Twilio integration for SMS and WhatsApp notifications                                            | Medium   |

## 15. Appendices

### Appendix A: COCO Vehicle Class IDs

| Class ID | Label      | Description                                |
|----------|------------|--------------------------------------------|
| 2        | car        | Sedan, SUV, hatchback, etc.                |
| 3        | motorcycle | Two-wheeled motorized vehicle              |
| 5        | bus        | Public transport bus, school bus           |
| 7        | truck      | Pickup truck, delivery truck, semi-trailer |

Note: Class IDs 0 (person), 1 (bicycle), 4 (airplane), 6 (train) are excluded from detection.

### Appendix B: YOLOv8 Model Variants

| Model                 | Size (MB) | Parameters | mAP@5 | Speed (CPU ms) | Used in TCS |
|-----------------------|-----------|------------|-------|----------------|-------------|
| YOLOv8n (Nano)        | 6.5       | 3.2M       | 37.3  | ~80ms          | Default     |
| YOLOv8s (Small)       | 22.5      | 11.2M      | 44.9  | ~130ms         | (Available) |
| YOLOv8m (Medium)      | 52.0      | 25.9M      | 50.2  | ~230ms         | (Available) |
| YOLOv8l (Large)       | 87.7      | 43.7M      | 52.9  | ~375ms         |             |
| YOLOv8x (Extra Large) | 136.7     | 68.2M      | 53.9  | ~480ms         |             |

### Appendix C: Project Directory Structure

```

TCS_Traffic_Congestion_System/
├── .git/                # Git repository
├── .gitignore           # Git ignore rules
├── README.md            # Project documentation
├── DESIGN.md            # UI design system specification
├── package.json         # Root monorepo config
├── requirements.txt     # Python dependencies
├── yolov8n.pt           # YOLOv8 Nano model weights (6.55 MB)
├──
├── # — Python Backend —————
├── main.py              # Standalone app (full visualization)
├── main_full.py         # Standalone app (ROI mode)
├── server.py            # Flask API + MJPEG streaming server
├── tracker.py           # YOLOv8 vehicle tracker wrapper
├── congestion_logic.py   # Threshold-duration congestion detector
├── location_service.py   # IP-based geo-location service
├── utils.py             # Drawing & geometry utilities
├── detector.py          # Simple detection-only module
├── test_camera.py       # Webcam connectivity test
├──
├── # — Data Output —————
├── congestion_images/    # Saved congestion evidence images
├── analytics_graphs/     # Saved vehicle count graphs
├──
├── # — React Frontend —————
├── frontend/
│   ├── package.json     # Frontend dependencies
│   ├── vite.config.ts   # Vite configuration
│   ├── tsconfig.json    # TypeScript config
│   ├── index.html       # HTML entry point
│   ├── src/
│   │   ├── main.tsx     # React app entry point
│   │   ├── App.tsx      # Main dashboard component
│   │   └── index.css     # Complete design system & styles
│   ├── dist/            # Production build output
│   └── node_modules/    # npm packages
├── legacy/              # Archived/legacy code
│   ├── congestion_images/
│   └── analytics_graphs/

```

## Appendix D: Glossary of Visual Overlays (Standalone Mode)

| Overlay              | Position              | Trigger           | Color                        | Purpose                              |
|----------------------|-----------------------|-------------------|------------------------------|--------------------------------------|
| Header bar           | Top (0,0 → w,60)      | Always            | Dark gray (#323232)          | Shows title and status               |
| Bottom panel         | Bottom (0,h-80 → w,h) | Always            | Dark gray                    | Vehicle count, threshold, clock, FPS |
| Bounding boxes       | Around vehicles       | Vehicle detected  | Green (normal) / Red (alert) | Vehicle identification               |
| Vehicle ID label     | Above each box        | Vehicle tracked   | Same as box                  | Track ID display                     |
| Heatmap overlay      | Over vehicle areas    | Vehicles present  | Red (40% alpha)              | Density visualization                |
| Congestion timer bar | Bottom (0,h-120)      | Count > threshold | Red                          | Progress toward congestion           |

|                |                    |                   |                            |                             |            |
|----------------|--------------------|-------------------|----------------------------|-----------------------------|------------|
| Warning banner | Top (0,60 → w,120) | Congested + blink | Red background, white text | "TRAFFIC<br>DETECTED"       | CONGESTION |
| ROI polygon    | Frame              | Always            | Yellow                     | Region of Interest boundary |            |
| FPS counter    | Bottom-right       | Always            | White                      | Performance monitoring      |            |
| Live clock     | Bottom-right       | Always            | White                      | Current time display        |            |

NOTE

This SRS document was generated based on analysis of the actual codebase of the TCS project. All code references, line numbers, and technical details are derived from the current implementation.

End of Document

Prepared by: Harsh Raj Date: August 8, 2026 Version: 1.0

End of Document

Prepared by: Harsh Raj  
Date: August 8, 2026 | Version: 1.0